

OS_2020-01-27

1 Synchronisation mit Semaphoren (30)

Ein Softwarepaket aus vier Prozessen soll eine Anlage simulieren, die Kartons mit Würfeln und Murmeln befüllt. Zwei gleichartige Prozesse simulieren den An- und Abtransport von Kartons zur bzw. von der Befüllanlage. Je ein Prozess kontrolliert das Einfüllen von Murmeln bzw. Würfeln in einen bereitstehenden Karton.

Ergänzen Sie die gegebenen Codestücke mit *Semaphoroperationen*, um die Prozesse entsprechend der unten gegebenen Bedingungen zu synchronisieren. Das Verwenden von globalen Variablen zur Synchronisation oder Kommunikation zwischen Prozessen ist nicht erlaubt.

- Die beiden Prozesse für den An- und Abtransport der Kartons sollen Roboter simulieren. Jeder Roboter hat einen eigenen Vorrat an leeren Kartons und einen eigenen Ablageplatz für volle Kartons und führt wiederholt folgende Aktionen durch:
 - Karton vom Vorrat holen: *karton_aufnehmen()*.
 - Karton zur Befüllanlage transportieren und öffnen: *karton_bereitstellen()*.
 - Karton verschließen und von der Befüllanlage nehmen: *karton_entfernen()*.
 - Karton zum Ablageplatz abtransportieren: *karton_abtransportieren()*.
- Der Prozess Murmelspender ruft in einer Endlosschleife die Funktion *Murmel_liefern()* auf. Jeder Aufruf dieser Funktion lässt eine Murmel in den Karton auf der Befüllanlage fallen.
- Der Prozess Würfelspender ruft in einer Endlosschleife die Funktion *Wuerfel_liefern()* auf. Jeder Aufruf dieser Funktion lässt einen Würfel in den Karton auf der Befüllanlage fallen.
- Die Prozesse sind so zu synchronisieren, dass die Parallelität ihrer Aktionen die folgenden Regeln erfüllt, darüberhinaus aber möglichst wenig eingeschränkt wird.
- Auf dem Befüllplatz der Anlage ist Platz für einen Karton. Synchronisieren Sie die Prozesse so, dass die Roboter die Befüllanlage abwechselnd mit Kartons versorgen.
- Jeder Karton ist mit N Murmeln und K Würfeln zu befüllen. Stellen Sie die korrekte Befüllung der Kartons durch die Synchronisation sicher.

(a) Initialisierungen (vor Start der Prozesse)

```

init(startR1, 1)      init(würfel, 0)
init(startR2, 0)      init(murmel, 0)

init(würfeldone, 0)
init(murmeldone, 0)

```

In der Folge sind Codestücke für die zu synchronisierenden Prozesse gegeben, die bereits alle notwendigen Funktionsaufrufe enthalten. Ergänzen Sie in diesen Codestücken die fehlenden Semaphoroperationen zur Synchronisation.

(b) Prozesse für die Befüllung

```
/** Murmelspender **/
```

```
do {
```

```
    wait(murmel)
```

```
    Murmel_liefern();
```

```
    signal(murmeldone)
```

```
} while(1);
```

```
/** Wuerfelspender **/
```

```
do {
```

```
    wait(würfel)
```

```
    Wuerfel_liefern();
```

```
    signal(würfeldone)
```

```
} while(1);
```

(c) Prozesse für Roboter

```
/** Roboter R1 **/
```

```
do {
```

```
/** Roboter R2 **/
```

```
do {
```

```
karton_aufnehmen(R1);
```

```
wait(startR1)
```

```
karton_bereitstellen(R1);
```

```
for (int i = 0; i < K; i++) { signal(würfel); }
for (int i = 0; i < N; i++) { signal(murmel); }

for (int i = 0; i < K; i++) { wait(würfeldone); }
for (int i = 0; i < N; i++) { wait(murmeldone); }
```

```
karton_entfernen(R1);
```

```
signal(startR2)
```

```
karton_abtransportieren(R1);
```

```
} while (1)
```

```
karton_aufnehmen(R2);
```

```
wait(startR2)
```

```
karton_bereitstellen(R2);
```

```
for (int i = 0; i < K; i++) { signal(würfel); }
for (int i = 0; i < N; i++) { signal(murmel); }

for (int i = 0; i < K; i++) { wait(würfeldone); }
for (int i = 0; i < N; i++) { wait(murmeldone); }
```

```
karton_entfernen(R2);
```

```
signal(startR1)
```

```
karton_abtransportieren(R2);
```

```
} while(1);
```

2 Page Replacement (30)

Gegeben ist ein Arbeitsspeicher mit vier Frames, dessen Seiten mit unterschiedlichen Ersetzungsstrategien ersetzt werden sollen: mit OPT, LRU, bzw. mit dem Clock Algorithmus. Ergänzen Sie in den Tabellen für jeden Algorithmus die Speicherinhalte für jeden Frame nach jedem Zugriff der angegebenen Seitenzugriffsfolge an. Die Seitenzugriffsfolge ist für alle Algorithmen gleich. Sie ist jeweils in der Kopfzeile der Tabelle gegeben. Geben Sie in den Spalten die Speicherbelegung nach dem entsprechenden Seitenzugriff an und kennzeichnen Sie in der letzten, mit *PF* markierten Zeile das Auftreten von Page Faults. Der Arbeitsspeicher ist am Beginn leer. Für die ersten vier Speicherzugriffe sind die Tabellen bereits ausgefüllt.

OPT-Strategie:

	A	B	C	D	C	E	D	B	A	F	E	D	A	F	D	A	E
0	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
1		B	B	B	B	B	B	B	B	F	F	F	F	F	F	F	F
2			C	C	C	E	E	E	E	E	E	E	E	E	E	E	E
3				D	D	D	D	D	D	D	D	D	D	D	D	D	D
PF	X	X	X	X		X				X							

LRU-Strategie:

	A	B	C	D	C	E	D	B	A	F	E	D	A	F	D	A	E
0	A	A	A	A	A	E	E	E	E	F	F	F	F	F	F	F	F
1		B	B	B	B	B	B	B	B	B	D	D	D	D	D	D	D
2			C	C	C	C	C	C	A	A	A	A	A	A	A	A	A
3				D	D	D	D	D	D	D	E	E	E	E	E	E	E
PF	X	X	X	X		X			X	X	X	X					

Clock-Algorithmus:

	A	B	C	D	C	E	D	B	A	F	E	D	A	F	D	A	E
0	A	A	A	A	A ₁	E ₁	E ₁	E ₁	E ₁	E ₀	E ₁	E ₁	E ₁	E ₁	E ₁	E ₁	E ₁
1		B	B	B	B ₁	B ₀	B ₀	B ₁	B ₀	F ₁	F ₁	F ₁	F ₁	F ₁	F ₁	F ₁	F ₁
2			C	C	C ₁	C ₀	C ₀	C ₀	A ₁	A ₁	A ₁	A ₁	A ₁	A ₁	A ₁	A ₁	A ₁
3				D	D ₁	D ₀	D ₁	D ₁	D ₁	D ₀	D ₁	D ₁	D ₁	D ₁	D ₁	D ₁	D ₁
PF	X	X	X	X		X			X	X							

Vergleichen und diskutieren Sie die Anzahl der bei den Seitenersetzungsstrategien beobachteten Page Faults. Entspricht das Ergebnis dem zu erwartenden Verhalten?

Das Opt das beste ist, ist zu erwarten, da es nur das beste sein kann, weil man in die Zukunft schaut, allerdings ist es nicht zu erwarten, dass der Clock Algorithmus, der nur eine Annäherung an LRU ist, besser abschneidet als LRU. Das ist hier so, weil wir viele Pages die wir gerade replaced haben direkt wieder gebraucht haben, was den Clock nicht betrifft.

Was versteht man unter einem *Process Image*? Erklären Sie, aus welchen Teilen ein Process Image besteht? (4)

Beschreibt derzeitigen Aufbau des Prozesses + wird vom OS für Memory Management gebraucht

Beinhaltet:

- Process control block
- User Data
- User Programm
- System Stack

Erklären Sie die Aktionen, die vom Betriebssystem bei einem *Process Switch* durchzuführen sind. Welche Arten von Ereignissen können zu einem Process Switch führen? (3)

Durchgeführt durch

- System Call
- Trap
- Interrupt

Aktionen die gemacht werden müssen:

- PCB abspeichern
- Abspeicherung von User und System Stack Pointern + CPU Register
- Austausch des Speicherraums
- TLB flushen
- Laden von allen genannten vom neuen Prozess

Nennen Sie Möglichkeiten, um in einem Paging-System Speicherschutz zu realisieren? (3)

- Bound Check
- Protection Keys
 - Jeder Frame hat einen Prot key, jeder Prozess hat key, bei adressierung wird abgeglichen
 - Jeder Prozess hat Menge von Keys, in TLB ist zu jeder logischen Adresse ein Key hinterlegt
-> Übereinstimmung geprüft
- Protection bits

Erklären Sie die Begriffe *absoluter Pfadname* und *relativer Pfadname* und geben Sie jeweils ein Beispiel an. (2)

Absolut beginnt bei root und startet mit /: Beispiel: /etc/file.txt

Relativ beginnt bei aktivem Ordner und startet ohne /: Beispiel fornite/file.txt

Was versteht man unter einem *Kernel Level Thread* und unter einem *User Level Thread*? Beschreiben Sie die beiden Arten der Threadimplementierung und charakterisieren Sie deren Unterschiede. (4)

Kernel sieht KLT, ULT sieht er nicht. Beide werden von API Libraries verwaltet aber von verschiedenen.

Unterschied beispielsweise in Pausierung... Wenn KLT Pausiert wird nur der Thread geblockt und bei ULT wird ganzer Prozess geblockt.

ULT - Management passiert im User-Space

KLT - Management passiert im Kernel

Nennen Sie die Bedingungen für das Eintreten eines Deadlocks und erklären Sie diese. (5)

- mutex -> nur gewisse Anzahl (meist 1) an Processes darf auf gewisse Ressourcen zugreifen
- Hold and wait -> Prozess kann Ressource halten und auf andere gleichzeitig warten
- no preemption -> man kann einem Process keine Ress wegnehmen
- circular wait -> Prozess A wartet auf R1 die PB gerade blockiert und PB wartet auf R2 die RA grade blockiert. -> gegenseitiges Warten

Bei der Realisierung von Dateisystemen gibt es verschiedene Möglichkeiten, um die zu einer Datei gehörenden Datenblöcke zu organisieren bzw. auffindbar zu machen (Block-Allokierung). Nennen Sie vier verschiedene Strategien zur Block-Allokierung von Dateien und beschreiben Sie diese mit ihren Vor- und Nachteilen. (5)

- Continuous Allocation: einfach alles ein großer block
 - v: schnelles traversieren
 - n: man kann schwer größe ändern
- Chained a: aufgeteilt in blöcke und mit pointern verbunden
 - v: größe änderbar
 - n: langsames traversieren weil alles aufgeteilt
- indexed: pointer in einem table zu den verteilten blöcken
 - v: random access schnell
 - n: viel speicherverbrauch im ram
- i-nodes: jede datei hat i-node mit informationen wie metadaten / wo die anderen blöcke sind
 - v: inode wird nur geladen, wenn man die datei verwenden will
 - n: man kann nur begrenzte pointer reinspeichern weil nicht so viel speicherplatz -> mehrere inode schichten

Nennen Sie vier Designprinzipien für die Konstruktion von sicheren Systemen. Geben Sie für jedes Prinzip ein Beispiel an. (4)

- open design: open source projekte
- least Privilegs: ein einfacher User darf nicht die kernel Dateien löschen
- default Einstellungen -> keine rechte: user soll von haus aus kein admin haben

- complete mediation: Bei jedem Zugriffsversuch schauen ob die Rechte vorhanden sind

Nennen Sie die Arten von Optimierungszielen, die ein Scheduler beim Prozess-Scheduling verfolgen kann und geben Sie jeweils Beispiele an. (4)

- Durchsatz:
 - Maximierung der Abgeschlossenen Jobs
 - SRT / FCFS
- Fairness
 - Vermeidung von Starvation
 - Round Robin
- Response Time
 - Antwort so schnell wie möglich
 - RR
- Einhalten von Deadlines
 - EDF
- Prozessorauslastung
 - Möglichst wenig Leerlauf, immer Arbeiten
 - Virtual Round Robin

Beschreiben Sie Aufgabe und Funktion eines *Translation Lookaside Buffers*? Worauf hat man bei der Betriebssystemimplementierung bei einem Process Switch zu achten, wenn man einen Translation Lookaside Buffer verwendet? (3)

Aufgabe: Hardware Cache zum Auffinden geladener Pages im Ram um die Adressübersetzung von virtuell auf physisch so schnell wie möglich zu machen.

Bei Process switch muss TLB gelöscht werden, da viele Prozesse die selben virtuellen Adressen verwenden, aber in den verschiedenen Prozessen der TLB auf verschiedene physische Adressen zeigen muss.

Erklären Sie die Begriffe *Access Control List* und *Capability List*. Wozu und wie werden diese verwendet? (3)

- **Access Control List (ACL):**
 - **Betrachtung:** Spaltenweise (objektzentriert).
 - **Erklärung:** Jedes *Objekt* (z. B. eine Datei) führt eine Liste, in der steht, welche Subjekte (User) welche Rechte (Read, Write, Execute) haben.
 - **Analogie:** Der Türsteher vor dem Club hat eine Gästeliste. Er prüft, ob du draufstehst.
 - **Verwendung:** Standard in Dateisystemen (NTFS bei Windows, ext4 bei Linux).
- **Capability List (C-List):**
 - **Betrachtung:** Zeilenweise (subjektzentriert).

- **Erklärung:** Jedes *Subjekt* (Prozess) besitzt eine Liste von "Tickets" oder "Schlüsseln" (Capabilities) für die Objekte, auf die es zugreifen darf. Wer das Ticket hat, darf zugreifen.
- **Analogie:** Du hast einen Schlüsselbund. Wenn du den Schlüssel für die Tür hast, kommst du rein (die Tür prüft nicht deine ID, nur den Schlüssel).
- **Verwendung:** Oft in verteilten Systemen, Microkernels (z. B. Mach, L4) oder mobilen Sandbox-Modellen (Android Permissions funktionieren ähnlich wie Caps).